

Metodología I

Unidad 8

Calidad – Análisis de Resultados y Acciones Correctivas

Nombre: Farias, Gustavo

Comisión: M2025-13

Matrícula: 101662

Repositorio GitHub:

https://github.com/Lucenear/Tecnicatura_Programacion_UTN/tree/main/3er_Semestre/Metodologia_I

CASO 1: "Reserva duplicada en horario ocupado"

Paso 1 - Interpretación del resultado

La evidencia muestra que el sistema permite registrar dos turnos idénticos (mismo médico, fecha y hora) para un mismo usuario sin emitir ninguna alerta ni bloqueo. Estamos ante un defecto funcional crítico. No es un error humano del tester ni una falla de infraestructura; es un bug de lógica de negocio donde la validación de disponibilidad no se ejecuta correctamente al momento de confirmar la segunda reserva.

Paso 2 - Clasificación del defecto

- Tipo: Error lógico.
- Justificación: El problema radica en la falta de una condición de validación previa a la inserción en la base de datos. El sistema procesa la solicitud como válida porque no verifica si ya existe un registro activo para esa franja horaria específica antes de ejecutar el INSERT. Es un fallo en la regla de negocio "un turno = un paciente por horario".

Paso 3 - Severidad y prioridad

- Severidad: Crítico.
- Prioridad: Alta.
- Justificación: La severidad es crítica porque corrompe la integridad de la agenda médica, generando conflictos reales de atención (dos pacientes esperando al mismo médico a la misma hora). La prioridad es alta porque debe solucionarse antes de cualquier despliegue a producción; permitir reservas duplicadas inutiliza la función principal del sistema y genera reclamos inmediatos.

Paso 4 - Propuesta de acción correctiva

- Backend: Implementar una validación transaccional en el endpoint /reservas que consulte la tabla de turnos antes de insertar. Si existe un registro activo para [medico_id, fecha, hora], rechazar la petición con código HTTP 409 (Conflict).
- Base de Datos: Agregar una restricción única (UNIQUE CONSTRAINT) compuesta por las columnas (medico_id, fecha, hora) para evitar duplicados incluso a nivel de concurrencia extrema.
- Frontend: Deshabilitar visualmente los horarios ya reservados en el calendario para evitar que el usuario intente seleccionarlos.
- Criterio de aceptación: Al intentar reservar un horario ocupado, el sistema debe responder con mensaje "Horario no disponible" y NO crear registro en BD.

Paso 5 - Análisis de Causa Raíz (5 Porqués)

1. Por qué se creó un turno duplicado? Porque el sistema permitió dos peticiones POST exitosas para el mismo slot temporal.
2. Por qué permitió la segunda petición? Porque no encontró un registro existente que bloqueara la operación.
3. Por qué no encontró el registro existente? Porque la consulta de verificación de disponibilidad no se ejecutó dentro de la misma transacción de reserva.

4. Por qué no estaba en la transacción? Porque la lógica de validación y la de inserción estaban desacopladas en el código.
5. Por qué estaban desacopladas? (Causa Raíz): Falta de diseño de pruebas de concurrencia y ausencia de revisión de arquitectura de datos durante el desarrollo del módulo de reservas.

Paso 6 - Plan de Re-testing

- Caso: Intentar reservar un turno ya confirmado.
- Datos: Médico: Dra. Molina | Fecha: 12/11/2025 | Hora: 10:00 (turno previamente ocupado por Usuario A).
- Resultado esperado: El sistema muestra mensaje "El horario seleccionado no está disponible" y NO se crea ningún nuevo registro en la base de datos.

Paso 7 - Plan de Regresión

1. Verificar que un usuario pueda reservar un horario libre correctamente (camino feliz).
2. Probar la cancelación de un turno y verificar que ese horario quede disponible inmediatamente para otro usuario.
3. Simular dos usuarios intentando reservar el mismo horario libre simultáneamente (prueba de concurrencia) para validar que solo uno obtenga el turno.

Paso 8 - Ticket

- ID: BUG-001
- Título: Reserva duplicada permitida en horario ocupado
- Resultado esperado: Bloqueo de reserva y mensaje de error al seleccionar horario ocupado.
- Resultado obtenido: Se confirma la reserva y se genera duplicado en BD.
- Estado: Fail
- Evidencia: Log [INFO] POST/reservas- usuario 182- OK (duplicado). Captura de pantalla de dos turnos idénticos en agenda.
- Tipo de defecto: Error lógico
- Severidad: Crítico
- Prioridad: Alta
- Propuesta de corrección: Validación transaccional + UNIQUE CONSTRAINT en BD + deshabilitar UI.
- Causa raíz: Lógica de validación desacoplada de la inserción y falta de pruebas de concurrencia.
- Plan de re-testing: Reservar horario ocupado → Esperar mensaje de error y 0 nuevos registros.
- Plan de regresión: Camino feliz de reserva, cancelación libera slot, prueba de concurrencia.
- Fecha de registro: 10/06/2026

CASO 2: "Formulario de registro acepta email inválido"

Paso 1 - Interpretación del resultado

El formulario acepta y guarda en base de datos un email con formato inválido (laura.perez@@gmail..com). Esto provoca que el flujo posterior de verificación de cuenta falle silenciosamente. Es un defecto de validación que impacta directamente en la capacidad del usuario de acceder al sistema tras registrarse.

Paso 2 - Clasificación del defecto

- Tipo: Error de validación.
- Justificación: El sistema carece de una expresión regular (regex) o librería de validación de emails tanto en frontend como en backend. Acepta caracteres prohibidos (@@, ..) que violan el estándar RFC 5322 de direcciones de correo.

Paso 3 - Severidad y prioridad

- Severidad: Alto.
- Prioridad: Alta.
- Justificación: Aunque el registro parece funcionar, el usuario queda imposibilitado de usar la cuenta (no recibe mail de verificación). Esto genera soporte técnico, frustración de usuarios y pérdida de conversiones. Técnicamente, ensucia la base de datos con registros inútiles.

Paso 4 - Propuesta de acción correctiva

- Frontend: Implementar validación en tiempo real con regex estándar de email al perder el foco del campo (onBlur) y al enviar el formulario. Mostrar mensaje claro: "Ingrese un email válido (ej: usuario@dominio.com)".
- Backend: Agregar validación server-side obligatoria antes de persistir. Rechazar con HTTP 400 si el formato es inválido. Nunca confiar en validación solo de frontend.
- Base de Datos: Limpiar registros existentes con emails inválidos mediante script de saneamiento.
- Criterio de aceptación: Emails con @@, .., sin @ o sin dominio son rechazados con mensaje descriptivo. Email válido permite registro exitoso.

Paso 5 - Análisis de Causa Raíz (5 Porqués)

- Por qué se guardó un email inválido? Porque ninguna capa del sistema rechazó el formato.
- Por qué no lo rechazó? Porque no hay validación de formato implementada.
- Por qué no hay validación? Porque el requisito de "email válido" se asumió implícito pero nunca se especificó técnicamente.
- Por qué no se especificó? Porque en el relevamiento no se definieron criterios de aceptación para campos de entrada.
- Por qué no se definieron criterios? (Causa Raíz): Ausencia de definición de reglas de validación en las historias de usuario y falta de pruebas de边界 (boundary testing) en formularios durante QA.

Paso 6 - Plan de Re-testing

- Caso: Registrar usuario con email inválido.
- Datos: Email: laura.perez@@gmail..com

- Resultado esperado: El formulario NO se envía. Se muestra mensaje de error debajo del campo email indicando formato incorrecto. No se crea usuario en BD.

Paso 7 - Plan de Regresión

1. Verificar que emails válidos estándar (usuario@gmail.com, nombre.apellido@empresa.ar) sigan siendo aceptados.
2. Probar otros formatos inválidos: sin @, sin dominio, espacios en blanco, caracteres especiales no permitidos.
3. Verificar que el mensaje de error sea accesible (lectores de pantalla) y claro para el usuario.

Paso 8 - Ticket

- ID: BUG-002
- Título: Formulario acepta email con formato inválido y bloquea verificación
- Resultado esperado: Rechazo de email inválido con mensaje de error descriptivo.
- Resultado obtenido: Registro exitoso con email laura.perez@@gmail..com y fallo silencioso en envío de verificación.
- Estado: Fail
- Evidencia: Log [WARN] Email validation skipped. Captura de formulario aceptando @@.
- Tipo de defecto: Error de validación
- Severidad: Alto
- Prioridad: Alta
- Propuesta de corrección: Regex en frontend + validación server-side + limpieza de BD.
- Causa raíz: Falta de criterios de aceptación para validación de inputs en historias de usuario.
- Plan de re-testing: Ingresar @@gmail..com → Esperar rechazo y mensaje de error.
- Plan de regresión: Emails válidos aceptados, otros formatos inválidos rechazados, accesibilidad del mensaje.
- Fecha de registro: 10/06/2026

CASO 3: “Certificados PDF con acentos mal impresos”

Paso 1 - Interpretación del resultado

Los certificados PDF generados muestran caracteres corruptos (GonzÃ¡lez, RepÃºblica) en lugar de tildes y . Aunque el archivo se descarga correctamente, el contenido oficial es ilegible e institucionalmente inaceptable. Es un defecto de codificación/integración que afecta directamente la validez legal y la imagen del instituto.

Paso 2 - Clasificación del defecto

- Tipo: Error de integración.
- Justificación: El problema surge de la incompatibilidad entre la codificación de caracteres de la fuente de datos (UTF-8) y la librería/generador de PDF que no está

configurado para manejar UTF-8 correctamente. Es un fallo en la comunicación entre componentes del sistema (BD → Generador PDF).

Paso 3 - Severidad y prioridad

- Severidad: Crítico.
- Prioridad: Alta (Inmediata).
- Justificación: Contrario a la opinión del responsable ("no es grave"), este defecto invalida completamente el propósito del certificado. Un documento oficial con errores ortográficos no puede entregarse en un acto institucional. El costo de reimprimir 500+ certificados, el daño reputacional y la logística de reentrega hacen que esto sea crítico. Además, el log muestra [WARN] Fallback de fuente por incompatibilidad UTF-8, confirmando que el sistema sabe que algo anda mal pero lo ignora.

Paso 4 - Propuesta de acción correctiva

- Generador PDF: Configurar explícitamente la codificación UTF-8 en la librería de generación de PDF. Integrar una fuente compatible con caracteres latinos extendidos (ej: DejaVu Sans, Roboto).
- Backend: Asegurar que la conexión a BD y las respuestas API usen charset UTF-8.
- Validación: Implementar test automático que genere un PDF con caracteres especiales (á, é, í, ó, ú, ñ, ü) y verifique que el texto extraído coincida exactamente.
- Criterio de aceptación: PDF generado con nombre "María González" muestra exactamente "María González" sin corrupción de caracteres.

Paso 5 - Análisis de Causa Raíz (5 Porqués)

- Por qué los acentos se ven mal en el PDF? Porque la librería de PDF no renderiza correctamente caracteres UTF-8.
- Por qué no los renderiza? Porque está usando una fuente por defecto que no soporta latin-extended o tiene fallback incorrecto.
- Por qué usa fuente incorrecta? Porque no se configuró explícitamente la codificación ni la fuente al inicializar el generador.
- Por qué no se configuró? Porque las pruebas de generación de PDF solo usaron nombres sin caracteres especiales durante el desarrollo.
- Por qué solo se probaron nombres simples? (Causa Raíz): Ausencia de datos de prueba representativos (nombres reales con tildes/ñ) en el entorno de testing y falta de validación de salida de documentos oficiales como parte de los criterios de aceptación.

Paso 6 - Plan de Re-testing

- Caso: Generar certificado con nombre que contenga caracteres especiales.
- Datos: Estudiante: "José María Núñez" | Carrera: "Comunicación Visual"
- Resultado esperado: El PDF descargado muestra exactamente "José María Núñez" y "Comunicación Visual" con tildes y correctos. Sin warnings en log.

Paso 7 - Plan de Regresión

1. Generar certificado con nombre sin caracteres especiales (ASCII puro) para verificar que no se rompió el caso base.
2. Probar con todos los caracteres especiales del alfabeto español: á, é, í, ó, ú, ñ, ü, ç, ÿ.
3. Verificar que el texto fijo institucional ("República Argentina", "Mendoza") se muestre correctamente en todos los certificados.
4. Confirmar que el tamaño del PDF y la calidad de impresión no se degradaron tras el cambio de fuente.

Paso 8 - Ticket

- ID: BUG-003
- Título: Certificados PDF muestran caracteres corruptos (UTF-8) en nombres y textos oficiales
- Resultado esperado: PDF con caracteres latinos extendidos (tildes, ñ) legibles y correctos.
- Resultado obtenido: Texto corrupto (González, República). Log muestra warning de fallback UTF-8.
- Estado: Fail
- Evidencia: Captura de PDF con caracteres corruptos. Log [WARN] Fallback de fuente por incompatibilidad UTF-8.
- Tipo de defecto: Error de integración
- Severidad: Crítico
- Prioridad: Alta
- Propuesta de corrección: Configurar UTF-8 + fuente compatible en generador PDF + test automático de caracteres especiales.
- Causa raíz: Falta de datos de prueba representativos y ausencia de validación de documentos oficiales en QA.
- Plan de re-testing: Generar PDF con "José María Núñez" → Verificar texto correcto.
- Plan de regresión: Nombres ASCII, todos caracteres especiales españoles, texto fijo institucional, calidad de impresión.
- Fecha de registro: 10/06/2026

CASO 4: “Carrito recalcula mal total después de aplicar descuento”

Paso 1 - Interpretación del resultado

Al aplicar un descuento del 10%, el sistema calcula erróneamente un 20% de descuento (\$30.000 → \$24.000 en lugar de \$27.000). La UI muestra correctamente "10%", pero el cálculo backend aplica el doble. Es un error lógico de regresión introducido por un cambio reciente en el módulo de precios, con impacto financiero directo.

Paso 2 - Clasificación del defecto

- Tipo: Error lógico.
- Justificación: La fórmula de cálculo del total final está mal implementada o fue alterada inadvertidamente durante la modificación del módulo de precios. El valor

almacenado en BD (descuento_aplicado: 0.10) es correcto, pero el total_final: 24000 no corresponde matemáticamente. Es un bug de lógica aritmética.

Paso 3 - Severidad y prioridad

- Severidad: Crítico.
- Prioridad: Alta (Hotfix).
- Justificación: Impacto financiero directo: la empresa pierde dinero en cada compra con descuento. Además, genera inconsistencias contables y riesgos de auditoría. Al ser una regresión post-deploy, indica que el proceso de release falló en detectar un efecto colateral. Debe corregirse inmediatamente y revertirse si no hay fix rápido.

Paso 4 - Propuesta de acción correctiva

- Backend: Revisar y corregir la función de cálculo de totales. Asegurar que $\text{total} = \text{subtotal} * (1 - \text{descuento_porcentaje})$. Separar claramente la lógica de aplicación de descuento de la lógica de precios base.
- Testing: Agregar prueba unitaria específica para cálculo de descuentos con múltiples escenarios (10%, 20%, 50%, 0%).
- Auditoría: Script para identificar y corregir pedidos afectados en BD. Notificar a clientes afectados si es necesario.
- Criterio de aceptación: Subtotal \$30.000 + descuento 10% = Total \$27.000 exacto. Valor en BD coincide con cálculo esperado.

Paso 5 - Análisis de Causa Raíz (5 Porqués)

- Por qué el total es \$24.000 en vez de \$27.000? Porque la función de cálculo aplicó el doble del porcentaje de descuento.
- Por qué aplicó el doble? Porque la modificación reciente en el módulo de precios alteró la fórmula de descuento sin actualizar la lógica correspondiente.
- Por qué se alteró la fórmula? Porque el desarrollador tocó código compartido entre módulos de precios y descuentos sin aislar responsabilidades.
- Por qué no se detectó antes de deploy? Porque no había prueba automatizada de regresión para el cálculo de totales con descuentos.
- Por qué no había prueba de regresión? (Causa Raíz): Ausencia de suite de pruebas unitarias/integración para lógica de negocio crítica (cálculos financieros) y dependencia excesiva de testing manual post-cambios.

Paso 6 - Plan de Re-testing

- Caso: Aplicar descuento del 10% sobre subtotal conocido.
- Datos: Productos: A=\$10.000, B=\$8.000, C=\$12.000 | Código: DESCUENTO10 (10%)
- Resultado esperado: Subtotal: \$30.000 | Descuento: \$3.000 | Total Final: \$27.000. Valor en BD: total_final=27000, descuento_aplicado=0.10.

Paso 7 - Plan de Regresión

1. Verificar cálculo SIN descuento (subtotal = total) para asegurar que no se afectó el camino base.

2. Probar otros códigos de descuento (20%, 50%) para validar que la fórmula funciona correctamente con distintos porcentajes.
3. Verificar carrito con productos de diferentes categorías/IVA para descartar efectos colaterales de la modificación de precios.
4. Confirmar que el historial de compras anteriores no fue alterado por el fix.

Paso 8 - Ticket

- ID: BUG-004
- Título: Cálculo de total aplica doble descuento (20% en vez de 10%) post-modificación módulo precios
- Resultado esperado: $\text{Total} = \text{Subtotal} * (1 - \text{descuento}\%)$. Ej: $\$30.000 - 10\% = \27.000 .
- Resultado obtenido: Total = \$24.000 (aplica 20%). UI muestra 10% correctamente. BD registra descuento=0.10 pero total=24000.
- Estado: Fail
- Evidencia: Log [DEBUG] discount_percent=0.1 / total=24000. Captura de carrito mostrando discrepancia.
- Tipo de defecto: Error lógico
- Severidad: Crítico
- Prioridad: Alta
- Propuesta de corrección: Corregir fórmula de cálculo + prueba unitaria de descuentos + script de auditoría BD.
- Causa raíz: Modificación de código compartido sin pruebas de regresión automatizadas para lógica financiera.
- Plan de re-testing: Subtotal \$30k + 10% descuento → Total \$27k exacto en UI y BD.
- Plan de regresión: Sin descuento, otros %, productos con IVA distinto, historial de compras.
- Fecha de registro: 10/06/2026